PHYS4840: Math and Comp Methods II:
Prof. Joyce
Final Project Guidelines

From Syllabus:

Final Project (20%): Students will propose and design their own software tool that solves a physics problem. The tool must use a technique we covered in class. Students will host their tool on a platform like GitHub or PyPI. Code documentation and usability of the software will be emphasized in scoring. Students will have 3 weeks to complete the project in lieu of homework assignments, with a deadline of Friday, May 9th. Students should discuss the suitability of their topic with me before working seriously on it—do not spend 80 hours writing a program before asking me if the proposed project is adequate!

Project Proposal (Due: Thursday April 24th)

You must submit a 1-paragraph summary of your intended project by next class (Thursday, April 24th) **OR at a later date following discussion of your project idea with me**, including:

- The physical problem your tool addresses
- The numerical or computational method(s) you plan to use
- A brief note on your expected input/output formats

You must receive approval before proceeding. This is to ensure the scope and direction are appropriate.

1. Describe the physics problem

Your tool should solve a **generalized** physics problem. You must describe the physics problem ("use case"), including schematics or diagrams where necessary.
Examples:

- This tool finds the voltages at every node of a circuit using matrix diagonalization (e.g. exercise 6.1, but generalized)
- This tool integrates an arbitrary stellar density profile using the 4th-order Runge-Kutta method
- This tool determines the optimal (fastest) data loading method for certain types of data/files
- This tool constructs an optimal combination of baked goods with exact change

2. Write code that is usable by others

- Create a README file that clearly explains what your tool is and how to use it. A README example can be found at the bottom of `https://github.com/mjoyceGR/PHYS4840`
- Provide instructions for downloading, configuring, and launching your tool
- Include a **test case**. Your repository must contain at least one example input and expected output that demonstrates the tool's correct functionality. This test case should be executable from the command line and explained in the README.

3. Adhere to programming best practices

- Functions should be stored in a functions library that is imported into your main program (as we have been doing all semester).
- Plots or videos, if your code produces them, should be saved to files, not pop up during run time.
- Larger algorithms should be built up from small, self-contained pieces. The fewer arguments a function takes, the better. Remember that functions can be called within other functions, or even passed as arguments themselves.

4. Document your code

Self-explanatory.

5. Host your code on GitHub (or PyPI) and provide working installation instructions

Your repository must include a clear list of dependencies and install/run instructions for a terminal environment. Dependencies include Python libraries like `numpy`, `pandas`, etc. While almost all of the libraries we have used in class are "standard issue," it is still your responsibility to let a user know which packages are needed to use your tool.

You will submit your project on GitHub. This includes any pdfs containing diagrams or textual information. You can upload (push) pdfs and documents to GitHub the same way you upload code.

6. Software Environment

- Your tool must run in a terminal environment (Linux or Mac).
- Projects should be developed using Sublime Text or another text editor (not Jupyter notebooks), as we have done in class.
- You may use either Python 3 or Fortran. Submissions written in Fortran will receive additional credit due to increased difficulty.

7. Grading Criteria (100 points total):

- **Correctness and Functionality (30 pts):** Does the code run without errors? Does it return the expected output over a range of physically valid inputs? Are results internally consistent or benchmarked when possible?
- **Generality and Flexibility (20 pts):** Does the tool work for a wide range of inputs or conditions? Are input limitations clearly documented? Is the solution adaptable to similar physics problems?
- **Documentation and README (20 pts):** Does the README clearly explain what the tool does, how to install it, and how to use it? Are example inputs and outputs shown? Are dependencies and system requirements listed?
- **Code Quality and Modularity (15 pts):** Are the functions well-organized and separated into logical units? Is the code readable, with appropriate variable names and comments? Are there redundancies or unnecessary complexity?
- **Originality and Scope (10 pts):** Is the chosen problem nontrivial? Is the solution elegant, insightful, or creatively implemented?
- **Version Control and Hosting (5 pts):** Is the project hosted on GitHub (or PyPI)? Are commits meaningful and descriptive? Does the repository include all necessary files and installation instructions?

Rubric Notes:

- Your tool must be general, meaning it works for an arbitrary input or over a well-defined (and well-explained) range of inputs. Points will be deducted if the code fails for a parameter combination that your usage instructions/documentation **did not indicate** should be excluded.
- If the purpose of your tool is to find eigenvectors, it defeats the purpose of this project to import `find_eigenvectors` (or equivalent) from another library. You should be writing your own algorithms, not using built-in functions from Python libraries. Things like `log` are OK. If you are unsure which functions it is OK to use, ask me.

Advice:

- Poke around my GitHub (especially more recent projects) to get an idea of the standard of documentation, for example: `https://github.com/mjoyceGR/MESA2HYDRO`
- Test your project by having a classmate try to set it up on their machine before you turn it in.